

VLLM专题（三十七）—Python 多进程



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#)

1. 调试

有关已知问题及解决方法的信息，请参见故障排除页面。

2. 介绍

重要提示

源码引用是基于 2024 年 12 月编写时的代码状态。

在 vLLM 中使用 Python 多进程的复杂性来自于：

- 将 vLLM 作为库使用，且无法控制 vLLM 的代码
- 多进程方法与 vLLM 依赖项之间存在不同程度的不兼容

本文档将描述 vLLM 如何应对这些挑战。

3. 多进程方法

Python 的多进程方法包括：

- **spawn** - 创建一个新的 Python 进程。自 Python 3.14 起，这将是默认方法。
- **fork** - 使用 `os.fork()` 来分叉 Python 解释器。在 Python 3.14 之前的版本中，这是默认方法。
- **forkserver** - 启动一个服务器进程，在需要时为每个请求分叉一个新进程。

****权衡****

- **fork** 是最快的方法，但与使用线程的依赖项不兼容。
- **spawn** 与依赖项的兼容性更好，但当 vLLM 作为库使用时可能会出现问题。如果使用代码时没有 `__main__` 保护（即 `if __name__ == "__main__":`），那么当 vLLM 启动一个新进程时，代码将被意外地重新执行，这可能导致无限递归等问题。
- **forkserver** 会启动一个新的服务器进程，根据需要分叉新的进程。不幸的是，当 vLLM 作为库使用时，它与 spawn 方法有相同的问题。服务器进程作为一个新进程被创建，这将重新执行没有 `__main__` 保护的代码。

对于 spawn 和 forkserver 方法，进程不应依赖于继承任何全局状态，正如使用 fork 时的情况那样。

4. 与依赖项的兼容性

多个 vLLM 依赖项指出使用 `spawn` 是首选或必需的：

- [PyTorch 多进程文档 - CUDA 在多进程中的使用](#)
- [PyTorch 多进程文档 - 共享 CUDA 张量](#)
- [Habana PyTorch 文档 - 使用 Gaudi 进行 PyTorch 入门](#)

更准确地说，在初始化这些依赖项之后，使用 `fork` 存在已知问题。

5. 当前状态 (v0)

环境变量 `VLLM_WORKER_MULTIPROC_METHOD` 可用于控制 vLLM 使用的多进程方法。当前的默认方法是 `fork`。

[vllm-project/vllm](#)

当我们知道进程是由 `vllm` 命令启动时（因为我们拥有该进程），我们使用 `spawn` 方法，因为它具有最广泛的兼容性。

[vllm-project/vllm](#)

`multiproc_xpu_executor` 强制使用 `spawn` 方法。

[vllm-project/vllm](#)

还有一些其他代码中硬编码了使用 `spawn` 方法的地方：

[vllm-project/vllm](#)

[vllm-project/vllm](#)

相关的 PR:

[Pull Request #8823](#)

6. v1 版本中的先前状态

在 v1 引擎核心中，有一个环境变量 `VLLM_ENABLE_V1_MULTIPROCESSING` 用于控制是否使用多进程。该变量默认关闭。

[vllm-project/vllm](#)

当启用时，v1 的 `LLMEngine` 会创建一个新进程来运行引擎核心。

[vllm-project/vllm](#)

[vllm-project/vllm](#)

[vllm-project/vllm](#)

默认关闭的原因是上述提到的所有问题——与依赖项和使用 vLLM 作为库的代码的兼容性。

6.1 v1 版本中的更改

Python 的多进程机制并没有一个适用于所有场景的简单解决方案。作为第一步，我们可以让 v1 进入一种“尽力而为”的状态，选择最适合的多进程方法以最大化兼容性。

1. 默认使用 `fork`。
2. 当我们知道控制主进程时（通过执行 `vllm` 命令），使用 `spawn`。
3. 如果检测到 **CUDA** 已被初始化，则强制使用 `spawn` 并发出警告。我们知道 `fork` 会失败，因此这是我们能做的最佳选择。

在这种场景下，已知仍然会失败的情况是：将 vLLM 作为库使用的代码在调用 vLLM 之前初始化了 CUDA。我们发出的警告应指示用户添加 `__main__` 保护或禁用多进程。

如果发生这种已知的失败情况，用户将看到两条消息来解释发生了什么。首先，vLLM 会记录一条日志消息：

```
WARNING 12-11 14:50:37 multiproc_worker_utils.py:281] CUDA was previously
  initialized. We must use the `spawn` multiprocessing start method. Setting
  VLLM_WORKER_MULTIPROC_METHOD to 'spawn'. See
  https://docs.vllm.ai/en/latest/getting_started/debugging.html#python-multiprocessing
  for more information.
```

其次，Python 本身会抛出一个异常，并附带清晰的解释：

```
RuntimeError:
  An attempt has been made to start a new process before the
  current process has finished its bootstrapping phase.

  This probably means that you are not using fork to start your
  child processes and you have forgotten to use the proper idiom
  in the main module:

      if __name__ == '__main__':
          freeze_support()
          ...

  The "freeze_support()" line can be omitted if the program
  is not going to be frozen to produce an executable.

  To fix this issue, refer to the "Safe importing of main module"
  section in https://docs.python.org/3/library/multiprocessing.html
```

通过这些更改，v1 版本能够在大多数情况下选择最佳的多进程方法，同时通过警告和错误信息帮助用户解决已知的兼容性问题。

7. 考虑的替代方案

7.1 检测是否存在 `__main__` 保护

有建议提出，如果我们能够检测使用 vLLM 作为库的代码中是否存在 `__main__` 保护，我们或许能更好地处理这个问题。这个问题曾在 StackOverflow 上被一位库作者提出，他面临着相同的问题。

确实可以检测我们是否处于原始的 `__main__` 进程中，或者是后续的分叉进程中。然而，检测代码中是否存在 `__main__` 保护似乎并不简单。

因此，这个选项被认为是不可行的，已被放弃。

7.2 使用 `forkserver`

最初，`forkserver` 似乎是一个很好的解决方案。然而，它的工作方式在 vLLM 作为库使用时，带来了与 `spawn` 相同的问题。

7.3 始终强制使用 `spawn`

一种清理的方法是始终强制使用 `spawn`，并在文档中说明，当使用 vLLM 作为库时，必须使用 `__main__` 保护。不幸的是，这样做会破坏现有的代码，并使得 vLLM 更难使用，这与我们希望使 LLM 类尽可能简单易用的目标相悖。

因此，我们决定不强行推行这一做法，而是保留一定的复杂性，尽力确保功能正常运行。

8. 未来工作

我们可能希望在未来考虑一种不同的工作管理方法，以应对这些挑战。

我们可以实现类似 `forkserver` 的方法，但将进程管理器设计为我们通过运行自定义子进程和自定义入口点来启动（启动一个 `vllm-manager` 进程）进行工作管理。